

Modelio Alchemist

Construction automatisée de modèles UML à partir de spécifications textuelles

🎯 Le Lot 5 automatise la construction de modèles

Quatre tâches du rescrit CIR :

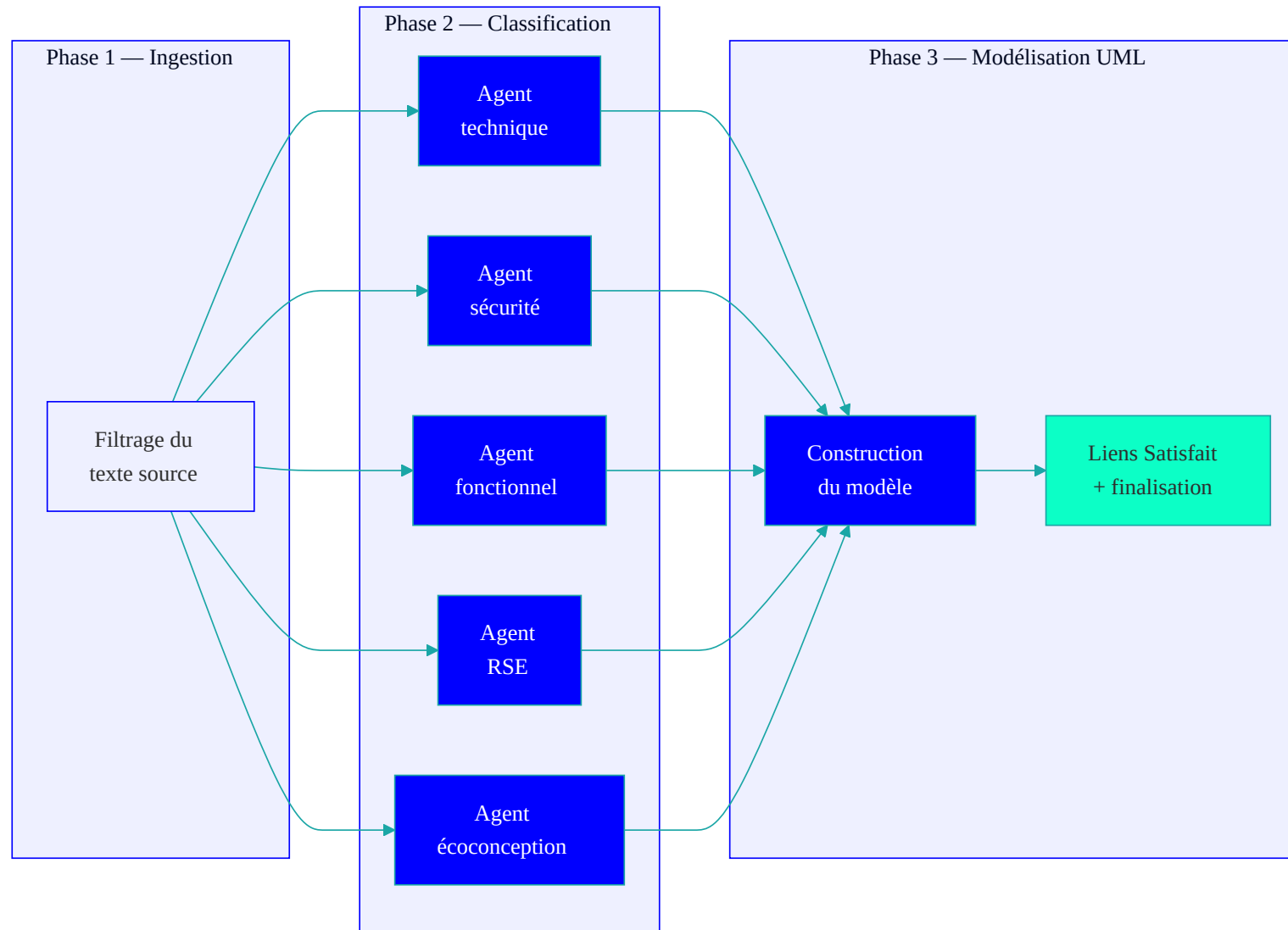
- **5.1** — Prototyper la création de modèles à partir de spécifications textuelles
- **5.2** — Adapter des modèles d'IA sur des données Modelio
- **5.3** — Intégrer les algorithmes dans ModelioBot, en conditions réelles
- **5.4** — Évaluer la qualité, la cohérence et la pertinence des modèles générés

Fondation 2025 — serveur MCP (Lot 2) :

Indicateur	Valeur
Outils UML exposés	45+
Couverture UML 2.5	92 %
Latence moyenne	87 ms
Gain PCA/PRA	8h → 12 min

2026 : le pipeline **Alchemist** construit un modèle UML complet à partir d'un cahier des charges — et une démarche empirique répond à la tâche 5.4.

Un pipeline en 15 étapes, 3 phases, 5 agents parallélisés



Un module Modelio, un pipeline Java autonome

Module Modelio (modelioalchemist)

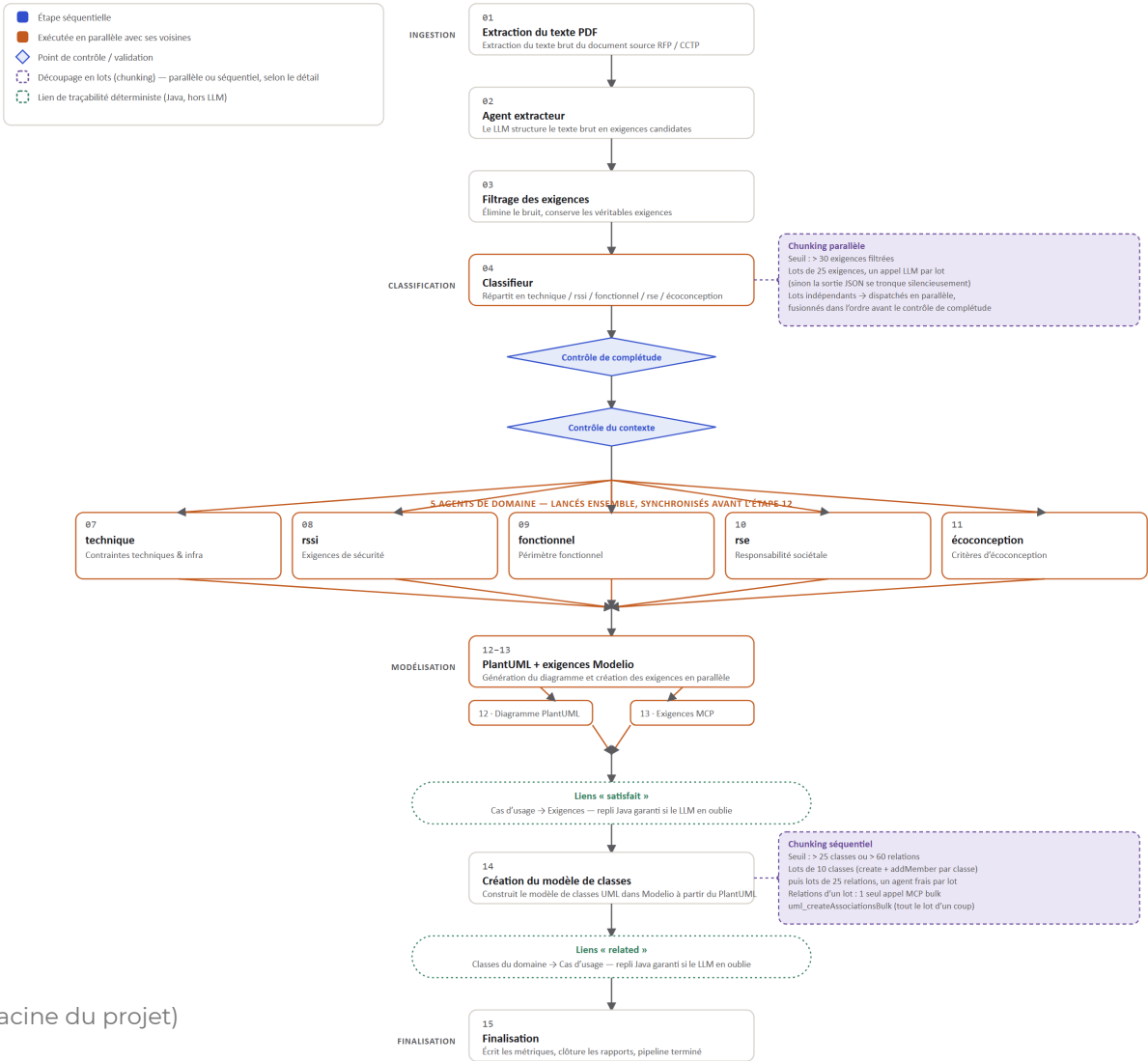
- Packagé en `.jmdac` via `modelio-maven-plugin` (API MDAKit 6.2.x, Java 17)
- Deux commandes contextuelles dans l'explorateur Modelio : `GenerateFromPdfCommand`, `GenerateExigencesForTMA`
- Chaque commande calcule un dossier de sortie projet (`modelioalchemist-output / tma-analysis-output`) et délègue au pipeline

Pipeline IA (`langchain/impl`, exécutable seul)

Dépendance	Rôle
<code>langchain4j-mcp</code> 1.4.0-beta10	Orchestration LLM + client MCP
<code>azure-ai-openai</code>	Modèle de langage (Azure OpenAI)
<code>pdfbox</code> 2.0.29	Extraction du texte source
<code>okhttp</code> / <code>jackson-databind</code>	Transport MCP, sérialisation

Le pipeline se lance aussi hors Modelio : `Main.main()` est un point d'entrée CLI pour rejouer un cahier des charges sans l'atelier.

Le pipeline en détail, tel qu'implémenté dans le code



La logique métier vit dans un seul package : `langchain/impl`

Classe	Responsabilité
<code>PipelineRunner</code>	Orchestre l'ingestion PDF, le filtrage et la classification (étapes 01-11)
<code>LangchainService</code>	Synthèse UML en 3 phases via un pool de <code>UmlModelingAssistant</code>
<code>DomainModelChunker</code>	Découpage en lots (exigences, classes, associations)
<code>ModelioMcpAgent</code> / <code>McpConnectionManager</code> / <code>McpRetryHandler</code>	Exécution MCP côté client — appels d'outils, reconnexion, retries
<code>RequirementsValidator</code> / <code>RequirementContextValidator</code>	Diagnostics de couverture (contrôle de complétude / de contexte)
<code>PipelineMetrics</code>	Instrumentation — écrit <code>pipeline_metrics.json</code> à chaque run
<code>TmaPipelineRunner</code> / <code>TmaRequirementsExtractor</code>	Variante spécialisée pour les dossiers de Tierce Maintenance Applicative

Chaque brique du récit R&D — traçabilité, chunking, repli déterministe, mesure — correspond à une classe nommée, pas à un script ad hoc.

Le code est sur GitHub, organisation Modelio-R-D

Accès

- Organisation : **Modelio-R-D**
- Dépôt : **ModelioAlchemist**
- Branche principale : `main`

```
git clone https://github.com/Modelio-R-D/ModelioAlchemist.git
```

Prérequis pour builder

- Java 17 · Maven 3.9+
- Modelio 6.1.1+ avec le module **Modelio MCP Server** actif
- Accès Azure OpenAI (`AZURE_OPENAI_AD_TOKEN`)

```
mvn clean package
```

 produit le module installable `ModelioAlchemist_x.x.x.jmdac` (dossier `modules/` de Modelio).

README à la racine du dépôt : capacités, variables d'environnement, utilisation CLI hors Modelio, structure du repository.

La traçabilité se joue en deux temps, du texte au modèle

Étage 1 — Extraction (étapes 01-04)

Le cahier des charges est décomposé en exigences (ex. EXG-001 à EXG-015), inscrites dans la table Modelio avec une colonne **Origin** : document, section, page, citation exacte.

Étage 2 — Construction (étapes 12-14)

Chaque élément UML créé (service, classe, entité) est relié à l'exigence qu'il réalise par un lien de dépendance **Satisfait**, standard de l'ingénierie des exigences.

Depuis un élément de diagramme → lien **Satisfait** → exigence → colonne **Origin** → section et page exactes du cahier des charges.

100 %

élément Modelio ↔ passage source, chaque maillon vérifiable

Ce mécanisme (Lot 5) est le symétrique des signets embarqués côté documents générés (Lot 3 / Lot 4).

Passé un seuil, le pipeline découpe le travail en lots

Un cahier des charges volumineux produit plus d'exigences ou de classes qu'un seul appel au modèle de langage ne peut traiter de façon fiable — au-delà d'un seuil, la sortie JSON se tronque **silencieusement**.

Étape	Seuil de déclenchement	Taille de lot	Mode
Classification (étape 04)	> 30 exigences filtrées	25 / appel	Parallèle, fusion ordonnée
Création des classes (étape 14)	> 25 classes ou > 60 relations	10 / lot	Séquentiel, agent frais
Création des associations (étape 14)	> 25 classes ou > 60 relations	25 / lot	Séquentiel, 1 appel MCP en masse

La création des classes reste séquentielle : la cohérence du modèle final dépend de l'ordre de création dans Modelio.

Ce qui se vérifie en code ne se confie pas au prompt

Avant correctif : le lien Satisfait était uniquement instruit par prompt, présenté comme optionnel.
Résultat mesuré : **0 lien créé sur 6 exécutions réelles**, alors que les identifiants nécessaires étaient disponibles à chaque fois.

Correctif : la décision de création est retirée au modèle de langage — une action comptable et déterministe s'exécute en code.
Résultat après correctif : **100 % de confirmation, 3/3 documents.**

Principe désormais systématisé aux Lots 1, 2 et 5 : repli déterministe pour toute action structurelle vérifiable mécaniquement.

La démarche empirique répond à la tâche 5.4

Échantillon — 5 documents non vus lors du développement, 3 vagues de mesure (31/07, 04/08, 05/08/2026) :

- CCTP AGIRC (retraite complémentaire)
- CCTP France Titres (titres sécurisés)
- CCTP Stationnement
- CCTP CDC 20245175
- CCTP RFP Conseil d'État

Méthode — chaque exécution produit un `pipeline_metrics.json` mesurant temps par étape, volumétrie des exigences, éléments UML créés, liens de traçabilité — **aucune valeur saisie manuellement.**

4 questions de recherche :

QR1 extraction/UML · QR2 traçabilité · QR3 volume · QR4 fiabilité de l'instrumentation elle-même

QR4 s'est imposée en cours de route : les premières mesures ont révélé que l'instrument de mesure contenait lui-même des défauts.

Trois défauts d'instrumentation corrigés avant toute conclusion de qualité

Vague	Constat
1 (31/07)	Liens Satisfait à 0 sur 3 runs · validation croisée non fonctionnelle · aucun comptage UML
2 (04/08)	Liens Satisfait toujours à 0 (6/6 cumulés) · défaut de validation croisée dépendant du document, pas systématique
3 (05/08, post-correction)	Liens Satisfait confirmés 100 % sur 3/3 · validation croisée cohérente · comptage UML complet

Le défaut de validation croisée n'était pas un échec constant du code : il dépendait de la structure du document source. Cela a permis de cibler le correctif sur la cause réelle plutôt que sur un symptôme.

QR1 à QR4 obtiennent chacune une réponse positive

QR1 — Extraction → UML

Positif avec réserve : 0 échec de création MCP, écart classifié/extrait de 3 à 11 requirements à investiguer.

QR2 — Traçabilité

Positif : liens Satisfait créés et confirmés à **100 %** sur 3 documents, contre 0 % avant correction.

QR3 — Volume

Positif sur la mesurabilité : **40-55 classes** et **42-62 associations** selon le document. Reproductibilité (runs répétés) non encore mesurée.

QR4 — Fiabilité de l'instrumentation

Positif : les 3 défauts identifiés sont corrigés et revérifiés sur le même échantillon avant/après.

100 %

confirmation des liens Satisfait, 3/3 documents post-correction

Les résultats sont crédibles mais leur portée reste bornée

Limites assumées :

- Échantillon de 3 à 5 documents : suffisant pour l'instrumentation, pas pour une généralisation statistique
- Pas de golden dataset : cohérence interne mesurée, pas l'exactitude sémantique face à une modélisation experte
- Un seul run par document par vague : reproductibilité non mesurée
- Résidu arithmétique mineur (filtering RFP) encore présent

Perspectives 2027 :

- Mesurer la reproductibilité (approche M3 du Lot 3 : 3 runs/cas)
- Étudier qualitativement les écarts de classification
- Étendre l'échantillon ($\approx 9-11$ min/document)
- Systématiser le repli déterministe comme grille de revue pour les Lots 1, 2, 5



Merci.

Questions ?

Juan Cadavid · juan.cadavid@docaposte.fr
Projet ModelioBot — CIR R&D · Lot 5 Alchemist